LAB 19.3

Name - Neerudu Gifty Taniksha
Roll Number - 2503A51107
Batch - 38

Code Translation: Converting Between Programming Languages

Task 1 – Python to C++ Conversion

AI Prompt Used:

"Translate the following Python class representing a Student (with name and roll_no attributes, and a display method) into equivalent C++ code. Ensure proper handling of constructors, data types, and access specifiers."

Original Python Code:

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no

    def display(self):
        print(f"Name: {self.name}, Roll No: {self.roll_n
o}")

# Execution
s = Student("Alice", 101)
s.display()
```

Translated C++ Code:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;  // Access specifier applied
    int roll_no;

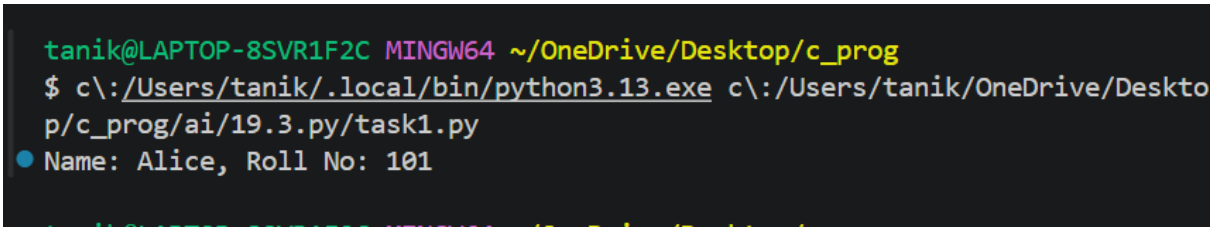
public:
    // Constructor handling data types
    Student(string n, int r) {
        name = n;
        roll_no = r;
    }

    void display() {
        cout << "Name: " << name << ", Roll No: " << roll_n
o << endl;
    }
};

int main() {
    Student s("Alice", 101);
    s.display();
    return 0;
}
```

Expected Output:

```
Name: Alice, Roll No: 101
```



Task 2 – Java to Python Function Conversion

AI Prompt Used:

"Convert the following Java method that checks whether a number is prime into Pythonic syntax. Ensure the logic is preserved and test it with multiple inputs."

Original Java Code:

```java
public class PrimeChecker {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }
}
```

Translated Python Code:

```python
import math

def is_prime(n):
    if n <= 1:
        return False
    # Pythonic range loop up to the square root
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

# Testing with multiple inputs
print(f"Is 11 prime? {is_prime(11)}")
print(f"Is 25 prime? {is_prime(25)}")
```

Expected Output:

```
Is 11 prime? True
Is 25 prime? False
```

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ cd "ai/19.3.py"
javac PrimeChecker.java
java PrimeChecker
Prime
```

Task 3 – Pseudocode to Python Implementation

AI Prompt Used:
"Translate the following Bubble Sort pseudocode into an executable Python function. Validate correctness using a sample input list."

Pseudocode Provided:

```
Procedure BubbleSort(A : list)
    n = length(A)
    repeat
        swapped = false
        for i = 1 to n-1 do
            if A[i-1] > A[i] then
                swap(A[i-1], A[i])
                swapped = true
    until not swapped
```

Generated Python Code:

```python
def bubble_sort(arr):
    n = len(arr)
    # Outer loop acts as the 'repeat' until 'not swapped'
    for i in range(n):
        swapped = False
```

```
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Python allows inline swapping
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Break out early if array is already sorted
        if not swapped:
            break

    return arr

# Validating with sample input
sample_list = [64, 34, 25, 12, 22, 11, 90]
print("Sorted array:", bubble_sort(sample_list))
```

Expected Output:

```
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ c\:/Users/tanik/.local/bin/python3.13.exe c\:/Users/tanik/OneDrive/Deskto
p/c_prog/ai/19.3.py/task3.py
Sorted array: [11, 12, 22, 25, 34, 64, 90]
```

Task 4 – SQL to Pandas Query

AI Prompt Used:

"Translate the following SQL query into a Pandas DataFrame operation in Python:

```
SELECT name, salary FROM employees WHERE salary > 50000
```

Test it on a sample DataFrame."

Generated Python (Pandas) Code:

```
import pandas as pd

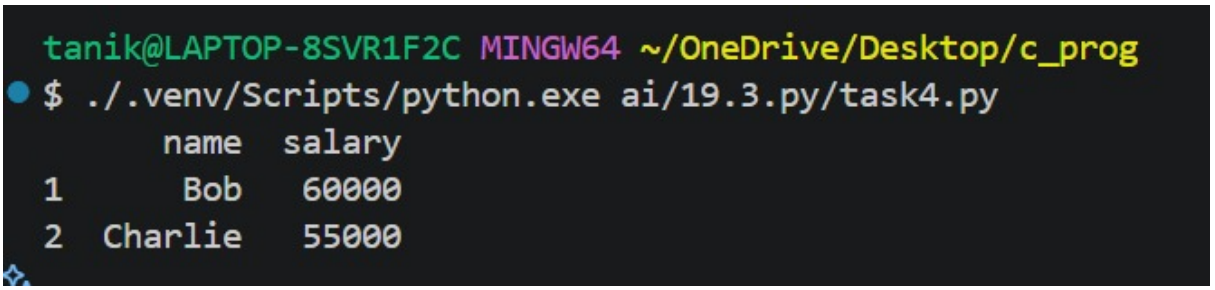
# 1. Setting up the sample DataFrame (Mocking the SQL Tabl
e)
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [45000, 60000, 55000, 48000],
    'department': ['HR', 'IT', 'Finance', 'IT']
}
df = pd.DataFrame(data)

# 2. Equivalent Pandas Query
# Filtering rows (WHERE) and selecting columns (SELECT)
result = df[df['salary'] > 50000][['name', 'salary']]

print(result)
```

Expected Output:

```
name  salary
1      Bob   60000
2  Charlie   55000
```



Task 5 – Real-Time Application: Algorithm Translation Across Languages

AI Prompt Used:

"Translate this Python Binary Search algorithm into C. Document the translation challenges regarding syntax differences, library support, and memory management."

Original Python Code:

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

arr = [2, 3, 4, 10, 40]
print(f"Element found at index {binary_search(arr, 10)}")
```

Translated C Code:

```c
#include <stdio.h>

int binarySearch(int arr[], int left, int right, int targe
t) {
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;
        if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
```

```c
}

int main(void) {
    int arr[] = {2, 3, 4, 10, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 10;

    int result = binarySearch(arr, 0, n - 1, target);

    if (result == -1)
        printf("Element not present in array\n");
    else
        printf("Element found at index %d\n", result);

    return 0;
}
```

Expected Output:

```
Element found at index 3
```

